

Answer Key — Streams, Friends & the Many Ways to Build an Object

Instructor reference for the review worksheet

array[0] Fill in the Blank

1. `ostream` (returns `std::ostream&`, enabling chaining)
2. `first`
3. `non-member`
4. `friend`
5. `default`
6. `overloading`
7. `member initializer list`
8. `declared` (declaration order in the class)
9. `member initializer list`

array[1] Find the Error

- Case A** Written as a *member*, `operator<<`'s left operand is `*this` — the `Fraction`. So it would only bind to `f << cout`; `cout << f` has no matching overload. Correct form is a non-member `ostream& operator<<(ostream& os, const Fraction& f)` (usually a `friend`).
DOES NOT COMPILE (`cout << f`)
- Case B** `f.num` and `f.den` are `private`; the free `operator<<` is outside the class and has no access. Fix: add inside `Fraction` — `friend ostream& operator<<(ostream&, const Fraction&);`
DOES NOT COMPILE
- Case C** Line 6: `PI = 3.14159`; assigns to a `const` member (which also was never initialized). A `const` member can only be set in the init list. Fix: `Circle(double radius) : PI(3.14159), r(radius) {}` with an empty body.
DOES NOT COMPILE
- Case D** `Fraction b(3)` — one `int` argument matches both `Fraction(int)` and `Fraction(int, int = 1)` equally well, so overload resolution is ambiguous. Fixes: delete `Fraction(int n)`; or collapse both into the single `Fraction(int n = 0, int d = 1) : num(n), den(d) {}`.
DOES NOT COMPILE
- Case E** The `friend` overload is fine. Inside the constructor, `x = x; y = y;` — name lookup finds the *parameters*, so each just assigns to itself and the members stay uninitialized. Fix: `Point(int x, int y) : x(x), y(y) {}` — in a mem-initializer `x(x)` is unambiguously *member(param)*.
COMPILES, PRINTS GARBAGE

array[2] Short Answer — model answers

1. *Why non-member:* In `cout << obj` the left operand is the stream (`std::ostream`), not your object. A member `operator<<` would take `*this` as the left operand, forcing the unnatural `obj << cout`. You cannot add a member to `std::ostream`, so the overload must be a free function taking `(ostream&, const T&)`.
2. *Return `ostream&` / friendship:* Returning the stream lets calls chain — `cout << a << b << endl;` parses as `((cout << a) << b) << endl;`, each `<<` feeding the next; with `void` the second `<<` has no stream on its left and won't compile. Friendship is safe here because `operator<<` only *reads* the state accessors would expose, is authored with the class, and adds no new mutation path.
3. *Declaration order:* Members initialize in the order they are *declared* (`size`, then `data`), regardless of list order — so `data(new int[size])` runs while `size` is still garbage and the array gets a garbage length. Fix: make sure `size` is the first-declared (and first-listed) member.
4. *Init list over assignment:* (1) It is the *only* way to set `const` members, reference members, members with no default constructor, and to pass args to a base-class constructor. (2) It initializes each member once directly; body assignment first default-constructs the member, then overwrites it — two steps, and sometimes impossible.